

API REST — Copa do Mundo 2026

Django REST Framework | Modelos, Relacionamentos e Endpoints

Disciplina	Semestre	Tipo	Valor Pontuação Extra
PABD — Banco de Dados	4º Semestre — TADS	Atividade de Fixação	

Contexto do Projeto

Nesta atividade você irá desenvolver, do zero, uma **API RESTful completa** para gerenciamento dos jogos da **Copa do Mundo FIFA 2026**. A competição reunirá 48 seleções divididas em 12 grupos de 4 times cada. Você deverá modelar as entidades, implementar os relacionamentos, criar os endpoints e validar o funcionamento com as ferramentas vistas em aula.

Regras da Copa do Mundo 2026
• Formato: 12 grupos (A a L) com 4 seleções cada — total de 48 seleções
• Classificação por grupo: 1º e 2º colocados + 8 melhores terceiros avançam
• Total de classificados: 32 seleções seguem para a fase eliminatória
• Fases eliminatórias: 32 avos → 16 avos (oitavas) → Quartas → Semifinal → Final
• Pontuação: Vitória = 3 pts Empate = 1 pt Derrota = 0 pt
• Critérios de desempate: 1º Saldo de gols 2º Gols marcados 3º Confronto direto

Especificação do Modelo de Dados

O sistema deve contemplar as entidades descritas a seguir. Atente-se aos relacionamentos e às regras de negócio de cada uma.

2.1 Entidade: Grupo

Campo	Tipo Django / Observação
nome	CharField(max_length=1) — Letra de A a L
descricao	TextField(blank=True) — Descrição opcional

2.2 Entidade: Selecao

Campo	Tipo Django / Observação
nome	CharField(max_length=100) — Nome completo da seleção
sigla	CharField(max_length=3, unique=True) — Ex.: BRA, ARG, FRA
confederacao	CharField com choices — UEFA, CONMEBOL, CONCACAF, AFC, CAF, OFC
grupo	ForeignKey(Grupo, on_delete=PROTECT, related_name='selecoes')
tecnico	OneToOneField(Tecnico, on_delete=SET_NULL, null=True, related_name='selecao')
escudo_url	URLField(blank=True) — URL opcional para imagem da bandeira/escudo

2.3 Entidade: Tecnico

Campo	Tipo Django / Observação
nome	CharField(max_length=150)
nacionalidade	CharField(max_length=100)
data_nascimento	DateField()

2.4 Entidade: Jogador

Campo	Tipo Django / Observação
nome	CharField(max_length=150) — Nome completo
nome_guerra	CharField(max_length=50) — Nome de guerra (apelido)
selecao	ForeignKey(Selecao, on_delete=PROTECT, related_name='jogadores')

posicao	CharField com choices — goleiro, zagueiro, lateral, volante, meia, atacante
numero_camisa	PositiveSmallIntegerField — de 1 a 26
data_nascimento	DateField()
suspenso	BooleanField(default=False) — Indica se o jogador está suspenso

2.5 Entidade: Jogo

Campo	Tipo Django / Observação
selecao_mandante	ForeignKey(Selecao, related_name='jogos_mandante', on_delete=PROTECT)
selecao_visitante	ForeignKey(Selecao, related_name='jogos_visitante', on_delete=PROTECT)
fase	CharField com choices — grupos, fase32, oitavas, quartas, semifinal, final
grupo	ForeignKey(Grupo, on_delete=PROTECT, null=True, blank=True) — Apenas na fase de grupos
data_hora	DateTimeField() — Data e hora do jogo
estadio	CharField(max_length=150, blank=True)
cidade	CharField(max_length=100, blank=True)
gols_mandante	PositiveSmallIntegerField(default=0)
gols_visitante	PositiveSmallIntegerField(default=0)
status	CharField com choices — agendado, em_andamento, encerrado, cancelado

2.6 Entidade: EventoJogo (Gols e Cartões)

Esta entidade registra cada evento ocorrido em um jogo: gols, cartões amarelos e cartões vermelhos. Permite rastrear quem fez o quê e em qual minuto.

Campo	Tipo Django / Observação
jogo	ForeignKey(Jogo, on_delete=CASCADE, related_name='eventos')
jogador	ForeignKey(Jogador, on_delete=PROTECT, related_name='eventos')
tipo	CharField com choices — gol, cartao_amarelo, cartao_vermelho, gol_contra
minuto	PositiveSmallIntegerField() — Minuto do evento
acrescimo	BooleanField(default=False) — Indica se ocorreu no tempo acrescido

Regras de Negócio dos Eventos

- Um jogador com 2 cartões amarelos no mesmo jogo deve ter suspenso=True automaticamente
- Um cartão vermelho direto também implica em suspenso=True para o próximo jogo
- Gols contra são contabilizados para a seleção adversária
- O campo minuto deve ser entre 1 e 120 (considera prorrogação)

Questões da Atividade

Questão 01 — Configuração do Projeto e Modelagem

Configure o projeto Django completo para a API da Copa do Mundo. Siga os passos abaixo:

1. Crie o projeto Django chamado `copa2026` e o app interno chamado `torneio`.
2. Configure o banco de dados PostgreSQL no `settings.py`, criando o banco `copa2026_db`.
3. Registre os apps `rest_framework` e `torneio` no `INSTALLED_APPS`.
4. Crie todos os modelos especificados na Seção 2 (Grupo, Selecao, Tecnico, Jogador, Jogo, EventoJogo) no arquivo `torneio/models.py`.
5. Execute `makemigrations` e `migrate` para criar as tabelas no banco.

Questão 02 — Serializers

Crie os Serializers no arquivo `torneio/serializers.py` conforme as especificações abaixo:

6. `GrupoSerializer` — todos os campos (`fields = '__all__'`).
7. `TecnicoSerializer` — todos os campos.
8. `SelecaoSerializer` — inclua o campo calculado `tecnico_nome` (`source='tecnico.nome', read_only=True`).
9. `JogadorSerializer` — inclua o campo `posicao_display` (`get_posicao_display`).
10. `EventoJogoSerializer` — inclua os campos calculados `jogador_nome` (`source='jogador.nome_guerra'`) e `tipo_display` (`get_tipo_display`), ambos `read_only`.
11. `JogoSerializer` — este é o mais complexo. Implemente com:
 - a) Campo `mandante_nome` (`source='selecao_mandante.nome', read_only=True`);
 - b) Campo `visitante_nome` (`source='selecao_visitante.nome', read_only=True`);
 - c) Campo `fase_display` (`get_fase_display, read_only=True`);
 - d) Serializer aninhado eventos (`EventoJogoSerializer, many=True`);
 - e) `SerializerMethodField` resultado que retorne 'Mandante venceu', 'Visitante venceu' ou 'Empate' com base nos gols;
 - f) Sobrescreva `create()` para inserir os eventos aninhados junto com o jogo.

Questão 03 — ViewSets, Filtros e Rotas

Crie os ViewSets no arquivo `torneio/views.py` e registre as rotas em `copa2026/urls.py`:

12. `GrupoViewSet` — `ModelViewSet` completo; sem filtros adicionais.
13. `TecnicoViewSet` — `ModelViewSet`; filtro por nome (`SearchFilter`).
14. `SelecaoViewSet` — `ModelViewSet` com filtros por grupo (`filterset_fields`) e busca por nome e sigla (`search_fields`).

15. JogadorViewSet — ModelViewSet com filtros por selecao, posicao e suspenso; busca por nome e nome_guerra; ordenação por selecao e numero_camisa. Otimize com `select_related('selecao')`.
16. JogoViewSet — ModelViewSet com filtros por fase, status e grupo; busca por estadio e cidade; ordenação por data_hora. Otimize com `select_related` e `prefetch_related` para os eventos e jogadores relacionados.
17. EventoJogoViewSet — ModelViewSet com filtros por jogo, jogador e tipo.
18. Registre todas as rotas no DefaultRouter com os prefixos: grupos, tecnicos, selecoes, jogadores, jogos, eventos-jogo.
19. Configure o Swagger (drf-yasg) para exibir a documentação em `/swagger/`.

Questão 04 — Endpoint de Classificação do Grupo

Implemente um endpoint personalizado que retorne a tabela de classificação de um grupo específico. Use o decorator `@action` do DRF no `GrupoViewSet`.

O endpoint deve responder em:

- GET `/copa2026_api/grupos/{id}/classificacao/`

A resposta deve ser uma lista ordenada das seleções do grupo com as seguintes informações calculadas a partir dos jogos encerrados:

Campo	Descrição
posicao	Posição no grupo (1º ao 4º)
selecao	Nome da seleção
sigla	Sigla de 3 letras
jogos	Total de jogos disputados
vitórias	Total de vitórias
empates	Total de empates
derrotas	Total de derrotas
gols_pro	Total de gols marcados
gols_contra	Total de gols sofridos
saldo	Saldo de gols (gols_pro - gols_contra)
pontos	Pontuação total (3×V + 1×E)

A ordenação deve seguir as regras oficiais da Copa: 1º pontos, 2º saldo de gols, 3º gols marcados. Exemplo de resposta esperada (HTTP 200):

```
GET /copa2026_api/grupos/1/classificacao/

HTTP 200 OK

[
  { "posicao": 1, "selecao": "Brasil", "sigla": "BRA", "jogos": 3,
    "vitórias": 3, "empates": 0, "derrotas": 0, "gols_pro": 7,
    "gols_contra": 1, "saldo": 6, "pontos": 9 },
  { "posicao": 2, "selecao": "Argentina", "sigla": "ARG", "jogos": 3,
    "vitórias": 1, "empates": 1, "derrotas": 1, "gols_pro": 4,
    "gols_contra": 3, "saldo": 1, "pontos": 4 },
  ...
]
```

Questão 05 — Endpoint de Controle de Fase

Implemente a lógica de controle e avanço entre as fases do torneio. Esta questão testa sua capacidade de integrar regras de negócio complexas à API.

5.1 — Endpoint: Estatísticas de um Jogo

Implemente via `@action` no `JogoViewSet`:

- GET `/copa2026_api/jogos/{id}/estatisticas/`

A resposta deve incluir:

- Placar: gols de cada seleção com lista de autores (`nome_guerra`, `minuto`);
- Cartões: lista de cartões amarelos e vermelhos por seleção;
- Resultado: 'Mandante venceu', 'Visitante venceu' ou 'Empate';
- Status atual do jogo.

5.2 — Endpoint: Avançar para Próxima Fase

Implemente via `@action` no `GrupoViewSet` ou em uma view separada:

- POST `/copa2026_api/jogos/avancar-fase/`

A action deve verificar se todos os jogos da fase de grupos estão encerrados e, se sim, calcular e retornar as 24 seleções classificadas (com base nas regras de classificação) e a fase em que cada uma entrou. A resposta não precisa criar os jogos da próxima fase automaticamente — apenas retornar a lista de classificados.

Estrutura sugerida da resposta:

```
POST /copa2026_api/jogos/avancar-fase/

HTTP 200 OK

{
  "fase_origem": "grupos",
  "fase_destino": "fase32",
  "classificados": [
    { "selecao": "Brasil", "sigla": "BRA", "grupo": "A",
      "posicao_grupo": 1, "pontos": 9 },
    { "selecao": "Argentina", "sigla": "ARG", "grupo": "A",
      "posicao_grupo": 2, "pontos": 6 },
    ...
  ]
}
```



```
],  
  "total_classificados": 24  
}
```

5.3 — Endpoint: Jogadores Suspensos

Implemente via `@action` no `JogadorViewSet`:

- GET `/copa2026_api/jogadores/suspensos/`

Retorna a lista de jogadores com `suspenso=True`, com o nome da seleção e o motivo da suspensão (acúmulo de amarelos ou cartão vermelho).

Roteiro de Testes

Após implementar todas as questões, teste a API usando pelo menos duas das ferramentas vistas em aula: Swagger UI, REST Client (VS Code) ou Insomnia/Postman. Abaixo estão os casos de teste mínimos.

4.1 Casos de Teste Mínimos

#	Ação	Método	Endpoint	Resultado Esperado
1	Criar Grupo A	POST	/grupos/	HTTP 201 — grupo criado
2	Criar Técnico	POST	/tecnicos/	HTTP 201
3	Criar Seleção BRA no Grupo A	POST	/selecoes/	HTTP 201
4	Criar Jogador	POST	/jogadores/	HTTP 201
5	Listar jogadores da seleção 1	GET	/jogadores/?selecao=1	HTTP 200 — lista filtrada
6	Criar Jogo (fase grupos)	POST	/jogos/	HTTP 201
7	Criar Evento de Gol	POST	/jogos/ (aninhado)	HTTP 201 com evento
8	Ver placar do jogo	GET	/jogos/1/estatisticas/	HTTP 200 com placar
9	Ver classificação do grupo	GET	/grupos/1/classificacao/	HTTP 200 com tabela
10	Listar jogadores suspensos	GET	/jogadores/suspensos/	HTTP 200
11	Deletar seleção com jogos	DELETE	/selecoes/1/	HTTP 403 ou 400 (PROTECT)
12	Busca textual	GET	/selecoes/?search=bra	HTTP 200 — filtrado

4.2 Arquivo api_copa.http (REST Client)

Crie o arquivo api_copa.http na raiz do projeto com o template a seguir. Adicione pelo menos os casos de teste 1 a 9 da tabela acima:

```
### Variáveis globais

@base = http://localhost:8000/copa2026_api
@contentType = application/json

#####

### GRUPOS

#####
```

```
### Criar Grupo A
POST {{base}}/grupos/
Content-Type: {{contentType}}

{ "nome": "A", "descricao": "Grupo da morte" }

###

### Listar grupos
GET {{base}}/grupos/
Accept: application/json

### ... (completar com os demais recursos)
```

Entrega e Critérios Gerais

5.1 O que deve ser entregue

- Repositório Git (GitHub/GitLab) com o código completo do projeto;
- Arquivo requirements.txt gerado com pip freeze;
- Arquivo api_copa.http com os testes do REST Client;
- Arquivo README.md com instruções de instalação e execução do projeto;